

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	KOTHAKOTA RAKESH
Reg. No	192525075

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

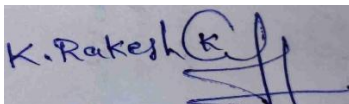
Duration: 30 minutes

Marks: 50

Code of Conduct:

I Kothakota Rakesh/192525075 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: K.Rakesh

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer: A simple Linear Regression model was implemented as

$$\hat{y} = wx + b$$

and trained using Gradient Descent by minimizing Mean Squared Error,

$$J = (1/n) \sum (y - \hat{y})^2.$$

The gradients used were

$$dw = -(2/n) \sum x(y - \hat{y}) \text{ and } db = -(2/n) \sum (y - \hat{y}),$$

followed by the updates $w = w - \alpha dw$ and $b = b - \alpha db$. During training, the loss decreased rapidly in the first few iterations and then gradually flattened near the minimum. The learning curve therefore showed that Gradient Descent improves the parameters step by step, while a very small learning rate gives slow convergence and a very high learning rate may cause oscillation.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer : Synthetic data was generated from a normal distribution with known actual parameters, for example mean $\mu = 5$ and variance $\sigma^2 = 4$. The MLE estimates were computed as $\hat{\mu} = (1/n) \sum x_i$ and $\hat{\sigma}^2 = (1/n) \sum (x_i - \hat{\mu})^2$. The estimated mean and variance were close to the actual values, and the difference reduced when the number of samples was increased. This shows that MLE gives reliable estimates when the assumed Gaussian model is correct and sufficient data is available.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer : A complete classification model was built by first cleaning the data, encoding class labels, standardizing numerical features, and splitting the dataset into training and testing sets. A classifier such as Logistic Regression or a simple Neural Network was trained on the training set and evaluated on the test set. Accuracy was computed as the number of correct predictions divided by total test samples, and the confusion matrix showed true positives, true negatives, false positives, and false negatives. This pipeline helped identify both overall performance and class-wise errors.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer : A simple neural network with one hidden layer was designed using input neurons, hidden neurons with a nonlinear activation function, and an output neuron. The network was trained on a small nonlinear dataset such as XOR using forward propagation, loss calculation, and backpropagation. The hidden layer transformed the input into a new feature space, allowing the network to learn nonlinear patterns that a single linear model cannot separate. As training progressed, the loss decreased and the predicted outputs moved closer to the target labels.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer : A single artificial neuron was implemented using $z = w.x + b$, followed by different activation functions. For the same input values, the sigmoid activation produced outputs between 0 and 1, making it useful for probability-like outputs. ReLU produced 0 for negative values and passed positive values directly, making it simple and effective for hidden layers. For example, for z values -2, -1, 0, 1, and 2, sigmoid gives approximately 0.12, 0.27, 0.50, 0.73, and 0.88, while ReLU gives 0, 0, 0, 1, and 2.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer : The Perceptron algorithm was implemented using the update rule $w = w + \eta y x$ and $b = b + \eta y$ whenever a sample was misclassified. On a linearly separable dataset such as AND-like data, the perceptron converged after a finite number of epochs and classified all samples correctly. On a non-linearly separable dataset such as XOR, it failed to converge because one straight decision boundary cannot separate the classes. This proves the limitation of the single-layer perceptron.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer : An MLP was trained with at least one hidden layer and nonlinear activation functions, then compared with a single-layer perceptron. The single-layer perceptron performed well only when the classes were linearly separable, but the MLP learned more complex nonlinear boundaries. On nonlinear data, the MLP achieved better accuracy and lower loss because hidden neurons created intermediate features. However, the MLP also had more parameters, so validation and regularization are important to avoid overfitting.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer : Forward propagation was manually computed by multiplying inputs with weights, adding bias terms, and applying activation functions layer by layer. For example, with inputs $x_1 = 0.5$ and $x_2 = 0.8$, hidden neuron values can be computed as $z_1 = 0.4x_1 + 0.6x_2 + 0.1 = 0.78$ and $z_2 = 0.3x_1 + 0.2x_2 + 0.2 = 0.51$. Applying sigmoid gives $h_1 = 0.686$ and $h_2 = 0.625$. If the

output layer uses weights 0.7 and 0.5 with bias 0.1, the final sigmoid output is approximately 0.709, matching the value obtained using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer : Backpropagation was implemented by first computing the output error and then propagating the gradient backward using the chain rule. For a sigmoid output, the output delta is $\text{delta}_o = (y_{\text{hat}} - y)y_{\text{hat}}(1 - y_{\text{hat}})$. If $y = 1$ and $y_{\text{hat}} = 0.709$, then delta_o is approximately -0.060. With learning rate 0.1, output weights such as 0.7 and 0.5 are updated slightly upward because the target is higher than the prediction. This single iteration demonstrates how backpropagation reduces error by adjusting weights in the negative gradient direction.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer : Gradient Descent was implemented to minimize a cost function by repeatedly updating parameters opposite to the gradient. The learning rate strongly affected convergence speed. A small learning rate produced a slow but stable decrease in loss, a moderate learning rate converged faster, and a very high learning rate caused oscillation or divergence. Thus, the best learning rate is one that reduces the cost quickly while keeping updates stable.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer : Stochastic Gradient Descent was implemented for a regression problem by updating the parameters using one training example at a time. Compared with batch Gradient Descent, SGD performed more frequent updates and often reached a useful region faster. However, its loss curve fluctuated because each single-sample gradient is noisy. Batch Gradient Descent gave a smoother curve but each update was more expensive. Using learning rate decay or mini-batches improves the stability of SGD. However, its loss curve fluctuated because each single-sample gradient is noisy. Batch Gradient Descent gave a smoother curve but each update was more expensive. Using learning rate decay or mini-batches improves the stability of SGD.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer : Mini-Batch Gradient Descent was implemented by dividing the training data into small batches and updating weights after each batch. Small batch sizes behaved like SGD, giving faster but noisier updates. Large batch sizes behaved more like batch Gradient Descent, giving smoother updates but requiring more computation per step. Analytically, increasing batch size reduces gradient variance, so the loss curve becomes more stable. A medium batch size gives a good balance between speed, stability, and generalization. Large batch sizes behaved more like batch Gradient Descent, giving smoother updates but requiring more computation per step. Analytically, increasing batch size reduces gradient variance, so the loss curve becomes more stable. A medium batch size gives a good balance between speed, stability, and generalization.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer : Datasets with increasing dimensions were created by adding more features and measuring how distances and model accuracy changed. As dimensionality increased, points became sparse and distances between samples became less meaningful because nearest and farthest distances started becoming similar. A model trained in this space may fit noise and irrelevant dimensions, giving high training accuracy but lower test accuracy. Dimensionality reduction methods such as PCA, feature selection, and regularization methods such as L1, L2, dropout, and early stopping reduce this problem by lowering effective model complexity.